



김광일 | Gwang-II Kim

🐱 Github : [kamothi](#)

📖 Blog : <https://blog.naver.com/kamothi>

✉ Email : rhkddlf7911@naver.com

🏆 Algorithm: <https://solved.ac/profile/rhkddlf7911>

안녕하세요, 개발자 김광일입니다.

저는 사람들과 이야기를 나누는 것을 즐깁니다.

다양한 도메인에 대해 의견을 공유하며 새로운 관점을 얻는 것을 좋아합니다.

이러한 소통을 통해 더 나은 해결 방법을 찾고,

최적화된 방안을 도출하는 과정에서 흥미를 느낍니다.

교육

건국대학교 컴퓨터공학부, 전체 학점: 3.86(4.5), 전공 학점: 3.88(4.5) 2019.03 - 2025.02(졸업)

경력

KICC(인턴) VAN 서비스팀 근무 2025.07 ~ 2025.09

- VAN 서비스 팀에서 백오피스 유지보수 및 개발 업무를 담당
- 해외간편결제와 세금환급서비스(TRS) 페이지에 대한 유지 보수와 개발을 진행

활동

SecurityFact(교내 보안 동아리) 2023.01 - 2023.12

Kuit(교내 IT 동아리) 2024.03 - 2024.08

2025 오픈소스 컨트리뷰션 아카데미 2025.07 ~ 2025.11

Kubernetes Organization Member 2025.10.21 ~

자격증

정보처리기사 2024.12.11

AWS Solutions Architect Associate 2025.01.25

기술

Java, Spring Boot, MySQL,

AWS(VPC, EC2, RDS), Docker, GitHub Action

Prometheus, Grafana

오픈소스 컨트리뷰션

github: <https://github.com/kubernetes/website>
opendev: [openstack-zuul-jobs](#), [openstacki18n](#), [openstack project-config](#)

Kubernetes/OpenStack한글화 컨트리뷰션, 국제화 및 커뮤니티 팀에서의 활동

Kubernetes에서는 현재 올라는 공식 문서를 기준으로 번역 및 업데이트 작업들을 진행하게 됩니다. 또한 번역을 위한 규칙을 업데이트하고 리뷰하는 활동들을 진행하고 있습니다. Openstack 팀에서는 기존에 Zanata라는 플랫폼에서 진행하던 번역 작업들을 Weblate로 마이그레이션하는 작업을 수행하고 있습니다. 저는 두 개의 파트 중 Openstack에 초점을 맞추어 작업을 진행하고 있습니다. Openstack 팀에서는 현재 기존 openstack 프로젝트의 CI/CD 작업과 Zanata 플랫폼을 분석하고 Weblate 오픈소스를 학습하여 100% 마이그레이션하는 것을 수행하고 있습니다.

역할

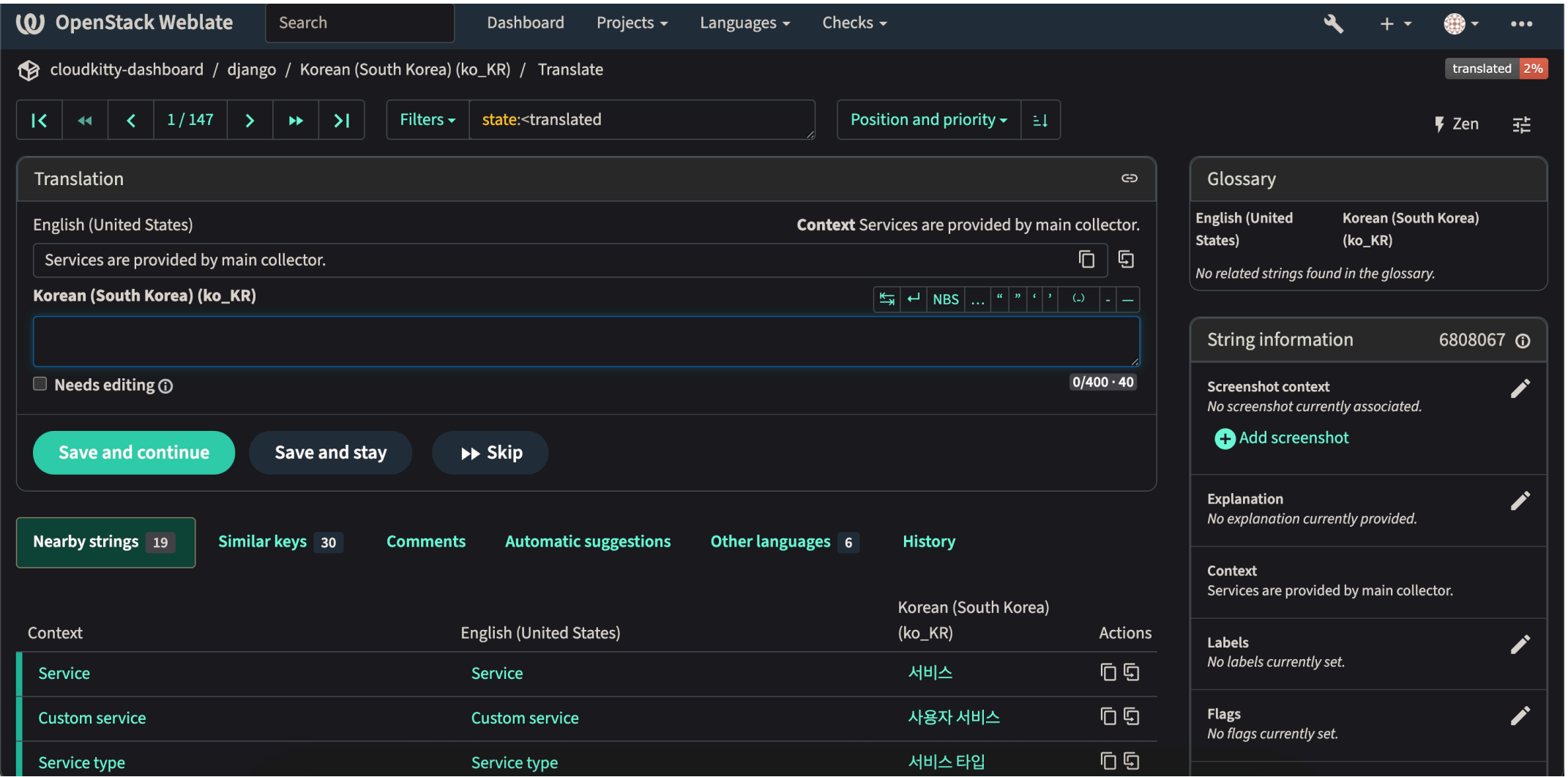
- openstack팀 리드멘티
- kubernetes 한글화 작업 참여
- weblate language 마이그레이션 작업
- upstream_translation_update_weblate.sh 개발(진행중)

기간

2025 07 ~ 2025.11

기술 스택

Python, Shell Script, MarkDown, Zuul, Ansible, Gerrit, Github



Weblate 화면

구현 일지 및 기록

- [오픈소스 컨트리뷰션 2주차](#)
- [오픈소스 컨트리뷰션 3주차](#)
- [오픈소스 컨트리뷰션 4주차](#)
- [오픈소스 컨트리뷰션 5주차](#)
- [오픈소스 컨트리뷰션 6&7주차](#)
- [오픈소스 컨트리뷰션 8주차](#)
- [오픈소스 컨트리뷰션 9&10주차](#)
- [오픈소스 컨트리뷰션 11주차](#)

오픈소스 컨트리뷰션

github: <https://github.com/kubernetes/website>

opendev: [openstack-zuul-jobs](#), [openstacki18n](#), [openstack project-config](#)

배경

openstack은 Kubernetes와는 다르게 zanata라는 오픈소스를 이용하여 해당 플랫폼에서 번역을 진행합니다.

Zanata: <https://translate.openstack.org/?dswid=2965>

현재 zanata라는 오픈소스는 업데이트를 멈춘 상태이며 2018.08 이후부터는 유지보수가 이루어지지 않고 있습니다. 버그가 발생시 버그 수정 또한 이루어지지 않습니다.

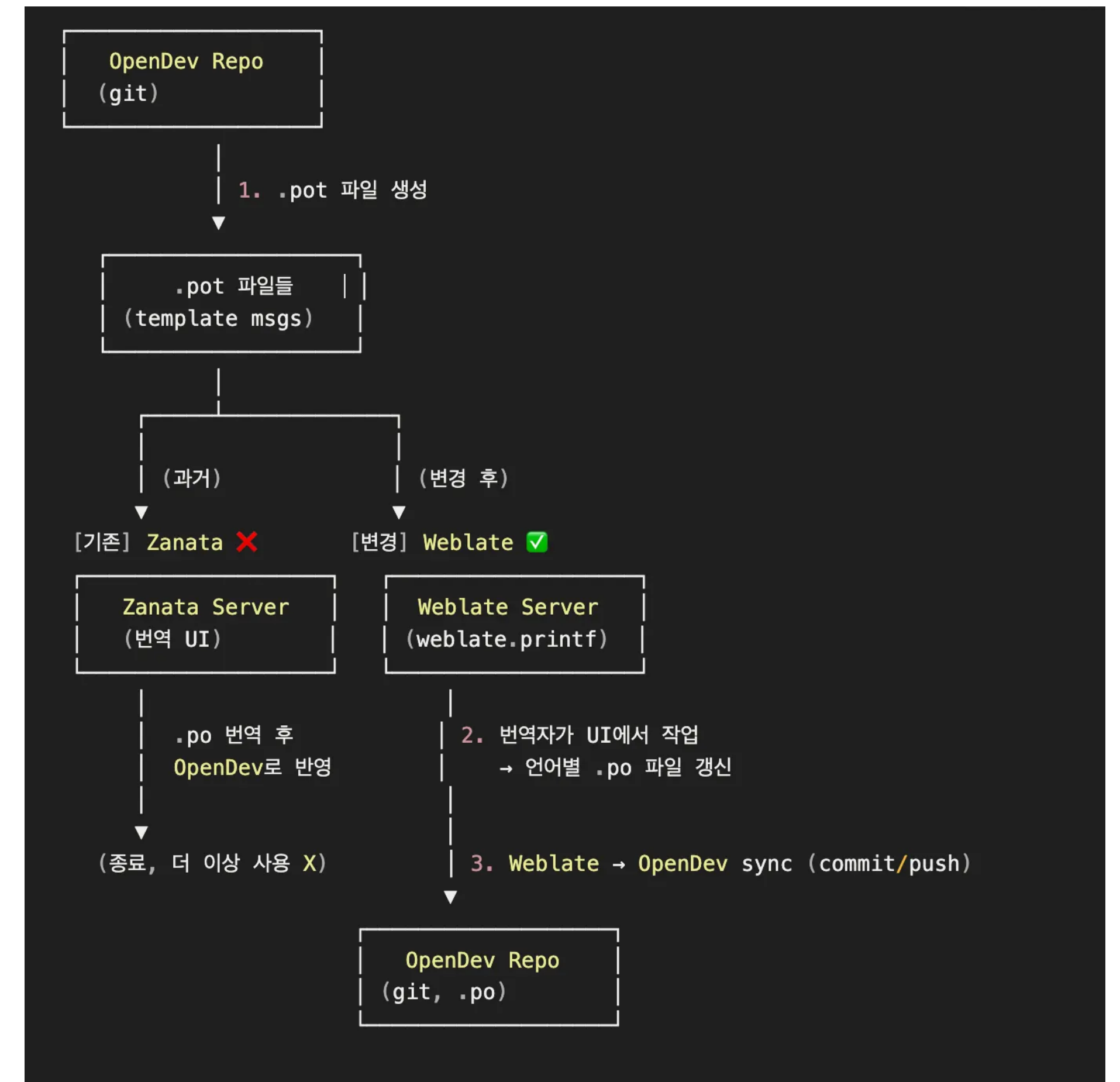
2022.04 OpenStack-i18n IRC 에서 Zanata → Weblate 마이그레이션 도입에 대한 언급이 이루어졌으며 현재 다른 플랫폼으로의 마이그레이션을 생각하여 Weblate라는 오픈소스 플랫폼을 이용하여 기존 Zanata에서 작업하던 내용들을 마이그레이션하는 작업을 수행하기로 결정하였습니다.

기존 워크 플로우

opendev에 프로젝트가 업데이트 되어 새로운 string들이 발생하면 그것들을 zuul이라는 ci도구와 Ansible을 활용하여 zanata로 pot파일을 생성하여 보내게됩니다. 그러면 zanata에서는 번역을 진행하고 번역한 내용들을 po파일을 생성시켜 opendev로 넘기게되는 과정을 거치게됩니다.

이러한 작업들을 기존 zanata에서 weblate로의 마이그레이션을 하는 것을 목표로 잡았으며 해야할 작업들은 아래와 같았습니다.

- 마이그레이션용 코드 ← Zanata의 Project 구조를 Weblate로 마이그레이션
- 최신 코드에서 문자열 업데이트(webplate) ← 기존 zanata 작업 마이그레이션
- weblate에서 번역된 문자열 프로젝트로 업로드 ← 기존 zanata 작업 마이그레이션
- 새로운 프로젝트 지원 ← 기존에 zanata에서는 수동으로하던 작업을 간편하게 지원
- 통계정보 추출 도구 개발



워크플로우 변화

오픈소스 컨트리뷰션

github: <https://github.com/kubernetes/website>
opendev: [openstack-zuul-jobs](#), [openstacki18n](#), [openstack project-config](#)

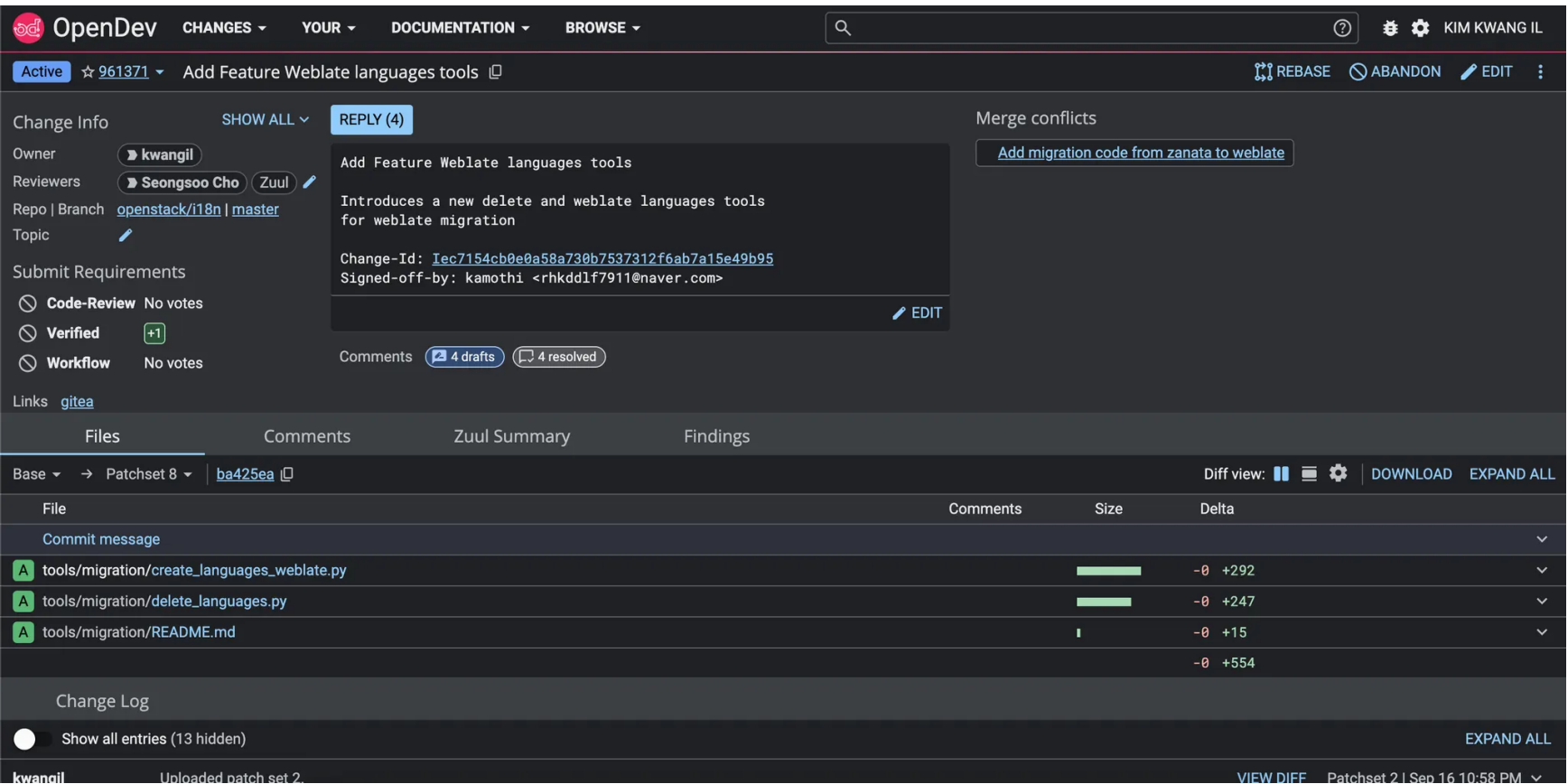
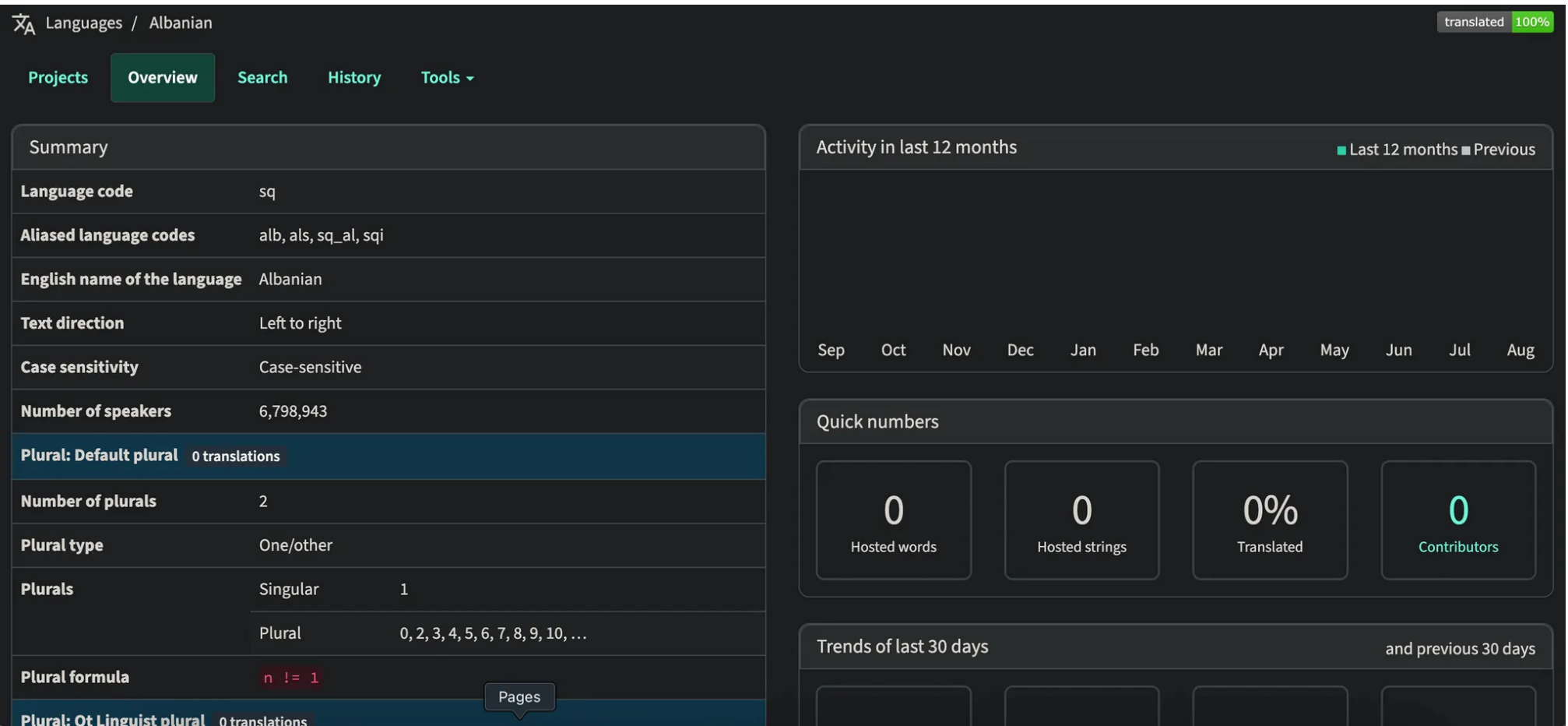
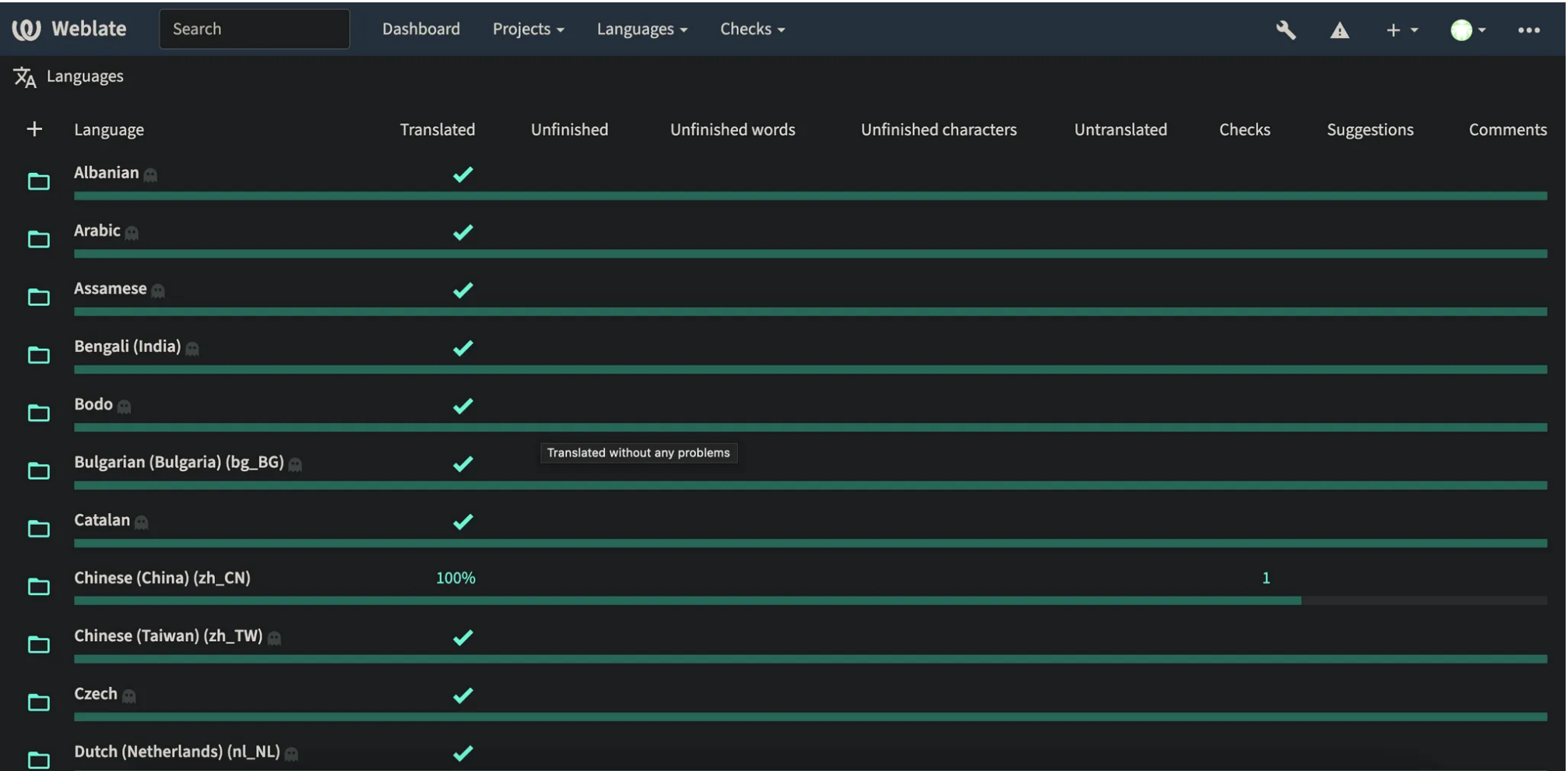
Language Migration

OpenStack의 번역 플랫폼을 Zanata에서 Weblate로 이전하는 과정에서, 두 플랫폼 간의 언어 규칙 불일치 문제가 발생했습니다. Weblate와 Zanata 모두 gettext 언어 규칙을 따르지만 Weblate를 처음 서버에 올렸을 때 생성되는 초기 언어들의 언어 포맷은 Zanata와 맞지 않아 마이그레이션 시 오류가 발생했습니다.

저는 단순히 UI로 수정하기보다, 실수를 줄이고 개발 이력을 남기기 위해 Python 코드로 언어 마이그레이션 로직을 구현했습니다. 먼저 Weblate의 기존 언어셋을 모두 삭제한 뒤, Zanata Git 저장소에서 언어 포맷 데이터를 불러와 OpenStack Zanata에서 사용 중인 언어셋과 매핑했습니다. 이후 누락된 언어 코드를 자동 생성하고 규칙을 추가하는 방식으로 두 플랫폼 간 언어 규칙을 일치시켰습니다.

결과적으로 OpenStack 번역팀의 Weblate 마이그레이션을 성공적으로 완료했습니다. 작성한 코드는 [opendev-i18n](#)에 패치로 반영되었으며, delete_languages.py로 weblate의 기존 언어를 정리하고, create_languages_weblate.py로 새로운 언어 세트를 생성하도록 구성했습니다.

링크: <https://review.opendev.org/c/openstack/i18n/+961371>



오픈소스 컨트리뷰션

github: <https://github.com/kubernetes/website>
opendev: [openstack-zuul-jobs](#), [openstacki18n](#), [openstack project-config](#)

upstream_translation_update.sh 만들기

기존 OpenStack에서는 프로젝트가 업데이트되면 Zuul이 CI 작업을 수행하며 upstream_translation_update.sh 파일을 호출해 번역 문자열(String)들을 Zanata 플랫폼에 업데이트했습니다. 하지만 Weblate로 마이그레이션을 진행하면서 기존 스크립트로는 Weblate 플랫폼에 업데이트를 수행할 수 없어, Weblate 환경에 맞는 새로운 번역 업데이트 로직이 필요했습니다.

이에 저는 Weblate 환경에 맞춘 upstream_translation_update_weblate.sh 파일을 새로 개발했습니다. 해당 스크립트는 프로젝트 업데이트 시 자동으로 새로운 번역 문자열을 Weblate로 반영하도록 구성했습니다. 또한 Zuul이 merge 이후 실행할 Job을 관리하고, 실제 작업은 Ansible Playbook에서 수행하기 때문에, Playbook에 Weblate 업데이트 task를 추가하여 OpenStack의 project-config 저장소에 패치셋을 반영했습니다.

결과적으로 Weblate 기반의 번역 자동화 파이프라인을 구성하고, 프로젝트 업데이트 시 번역 데이터가 자동으로 Weblate에 반영되도록 만들었습니다. 이를 통해 기존 Zanata 환경에서의 번역 관리 흐름을 Weblate로 완전하게 이전할 수 있는 준비를 하였습니다.

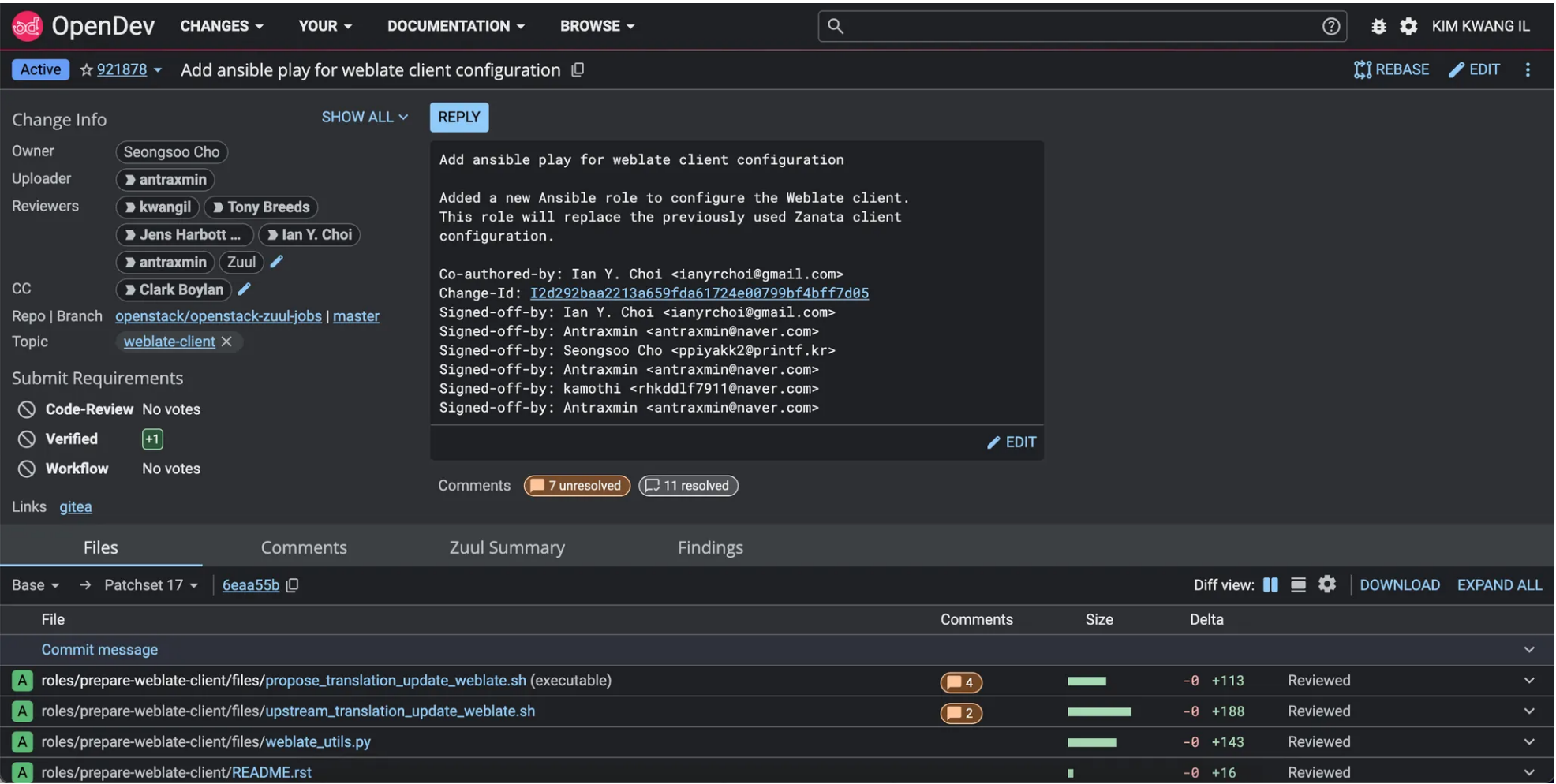
링크: <https://review.opendev.org/c/openstack/openstack-zuul-jobs/+/-/921878>

오픈스택 마이그레이션 작업 공유

오픈인프라 한국 커뮤니티 포럼과 오픈스택 wiki에 관련 글들을 작성하여 공유하여 지식 공유를 진행한 상태입니다. 포럼에는 Openstack Weblate Migration - upstream_translation_update_weblate.sh 과 Openstack Weblate Migration - Language Migration 라는 제목의 글로 공유하였으며 위키에는 마이그레이션 툴 페이지를 만들어 공유를 진행해나가고 있습니다.

포럼: [Openstack Weblate Migration - Language Migration](#), [Openstack Weblate Migration - upstream_translation_update_weblate.sh](#)

위키: [I18nTeam/Migration-to-weblate/migration-tools](#)



쿠버네티스 번역 작업

현재 5개의 Issue와 5개의 PR을 작성하여 올린 상태입니다. 해당 이슈는 쿠버네티스 공식 문서에 올라온 문서들 중 한글화 작업이 이루어지지 않은 문서들을 작업 내용들입니다. 또한 쿠버네티스 팀원들과 같이 피어 리뷰를 진행하며 서로의 번역 작업들을 피드백하고 개선 사항들을 공유하며 번역의 품질을 높여나가고 있습니다.

링크: <https://github.com/kubernetes/website/pulls?q=is%3Apr+author%3Akamothi+>

쿠타버스: 학교를 배경으로 한 메타버스 게임 제작

Github <https://github.com/KU-Taverse>

학교를 배경으로 한 메타버스 게임

학교라는 맵을 통해 유저들끼리 자유롭게 이동이 가능하고 각 건물들에 미니 게임을 할 수 있는 공간을 두어 사용자들 간 게임 대결이 가능합니다. 캐릭터의 이동, 유저 도메인, msa에 필요한 다른 서버들로 구성하였으며 추가적인 도메인 개발 시 해당 로직만 수정하여 개발할 수 있게 구성하였습니다. 배포는 AWS의 기능들을 활용하였으며 Github Action을 활용하여 ci/cd를 구축하고 ec2 4대를 활용하여 서버의 분산과 vpc를 활용한 내부 통신으로 구성하였습니다.

역할

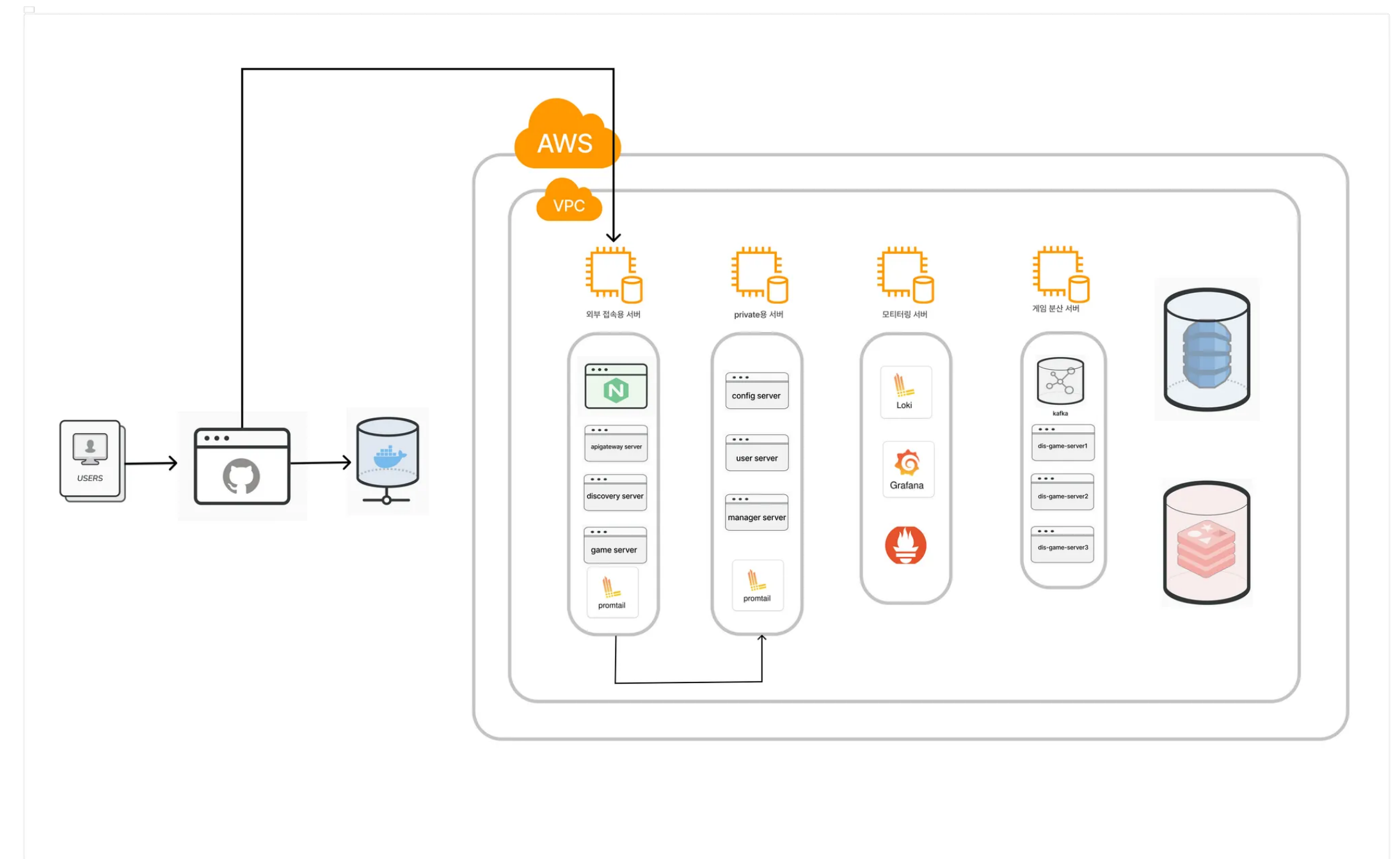
- 프로젝트 기획
- msa 시스템을 위한 서버들을 구축
 - Config Server
 - Apigateway Server
 - Discovery Server
- Spring Security를 이용한 소셜 로그인 구현
- 미니 게임 시스템 구현
- 시스템 아키텍처 설계 및 CI/CD 구축

기간

2024 03 ~ 2024.12

기술 스택

Spring Cloud, Spring Boot, MySQL, Spring Webflux,
Docker, AWS, Nginx, Grafana, Promtail, Prometheus, Kafka



아키텍처

구현 일지 및 기록

- 아키텍처의 전환
- 미니게임 분산 처리 도입기

쿠타버스: 학교를 배경으로 한 메타버스 게임 제작

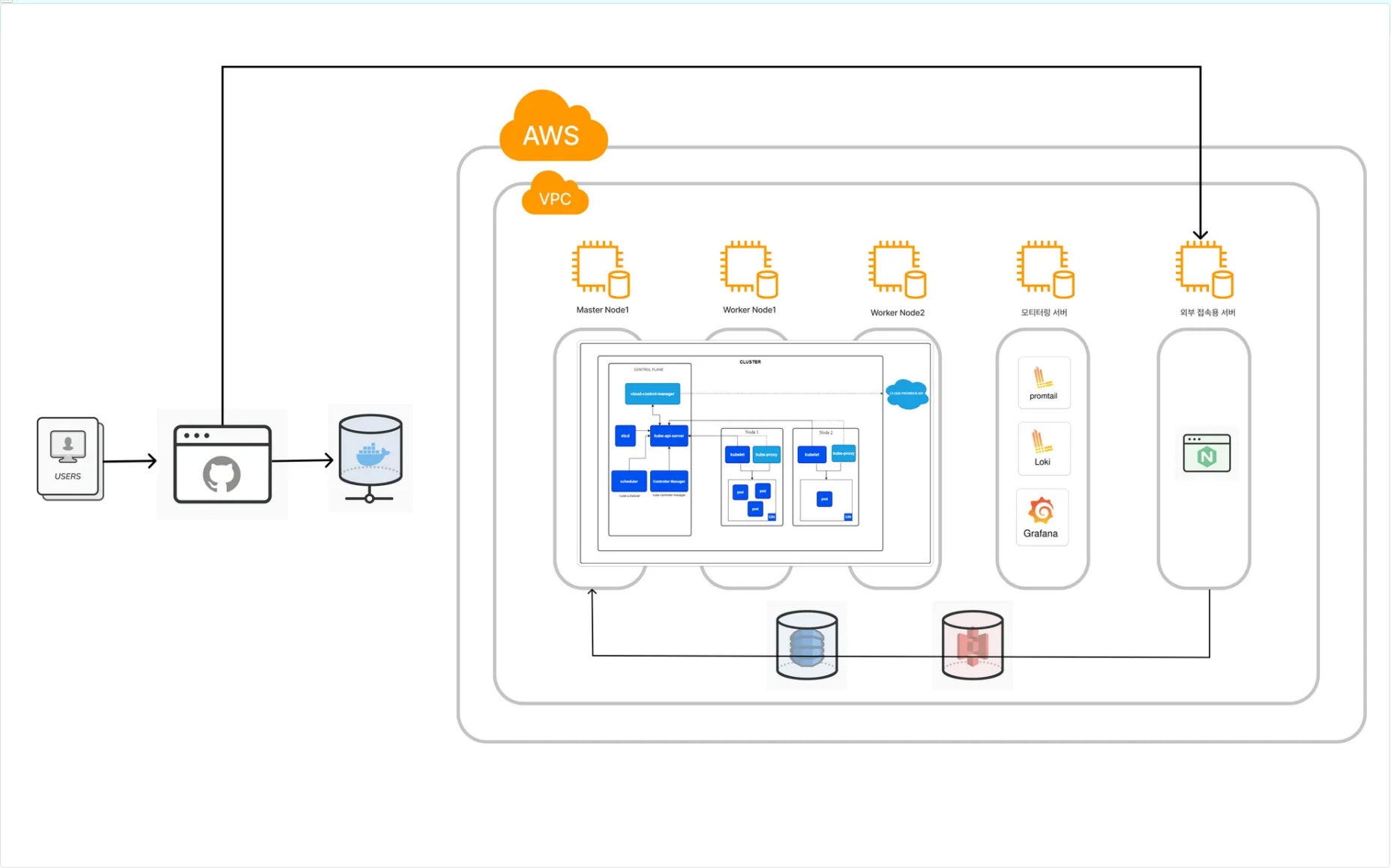
Github <https://github.com/KU-Taverse>

아키텍처 설계

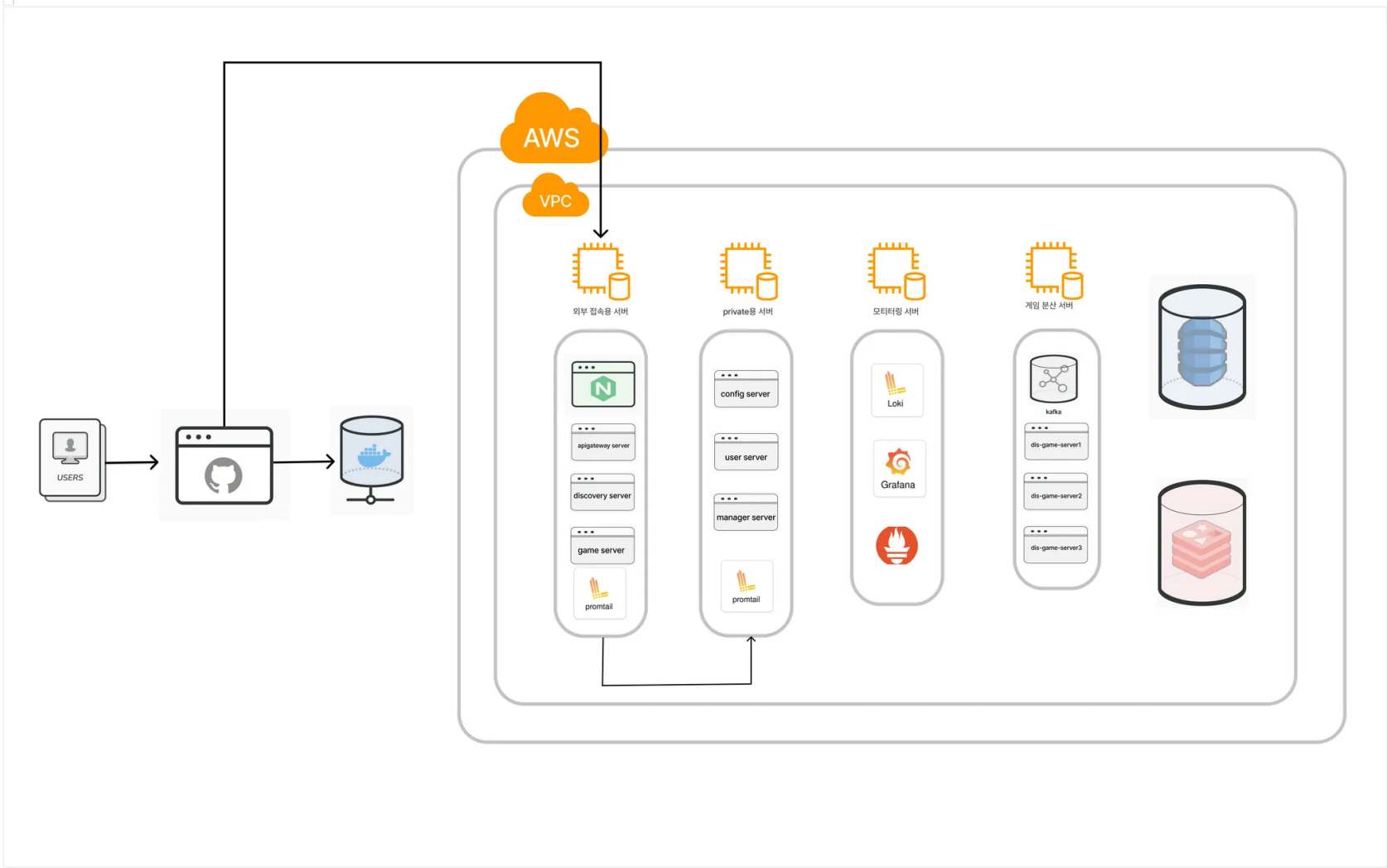
초기에는 하나의 VPC 내에서 내부 통신을 private IP로 처리하고, 외부 통신은 public IP를 가진 서버를 통해 수행하는 구조로 설계했습니다. MSA의 각 서버는 Kubernetes 파드로 배포하여 관리 효율성을 높이하고자 했습니다. 그러나 Kubernetes의 최소 사양 요구와 AWS 비용 부담이 커지면서, 지속 가능한 운영이 어렵다는 한계가 드러났습니다.

이에 따라 Kubernetes를 제거하고, 최소한의 EC2 인스턴스와 스토리지를 활용한 경량 아키텍처로 전환했습니다. 민감한 정보를 다루는 서버와 DB에는 public IP를 제거하고 private 네트워크 통신만 허용하여 보안성을 강화했습니다. 또한 API Gateway 서버가 위치한 EC2에는 Nginx를 구성하여 HTTPS 처리와 로드밸런싱을 수행하도록 하여, 요청 분산과 트래픽 안정성을 확보했습니다.

결과적으로 비용 효율적이면서도 안정적인 MSA 인프라를 구축할 수 있었습니다. Kubernetes 없이도 서비스 간 통신 구조를 단순화하고, Nginx 기반 로드밸런싱을 통해 트래픽 증가 상황에서도 안정적인 요청 처리가 가능해졌습니다. 다만 약 3개월간 운영 결과 약 230달러의 비용이 발생 하며, 여전히 개선 여지가 있음을 확인했습니다. 이를 통해 무조건 MSA 구조를 적용하기보다, 비즈니스 규모와 상황에 맞는 아키텍처를 설계하는 것이 더 중요하다는 점을 깨달았습니다.



초기 아키텍처



변경 아키텍처

쿠타버스: 학교를 배경으로 한 메타버스 게임 제작

Github <https://github.com/KU-Taverse>

미니게임

미니게임의 경우 초당 30번 데이터를 전달해야 하기에 부하가 발생하게 됩니다. 그렇기에 비동기적 처리에 특화된 spring webflux를 활용하여 구현하였습니다. 게임 서버의 경우 많은 데이터를 처리를 하기에 지속적인 연결을 유지해야 합니다. 일반적인 http 통신의 경우 stateless한 성격을 가지게 됩니다. 또한 단방향으로 통신이 되기에 양방향 통신을 위하여 websocket을 활용 하였습니다. 게임 매칭을 위한 대기열은 queue 자료 구조를 활용하여 구현하였으며 Scheduling을 통해 자동으로 매칭될 경우 Room을 만들어 게임 처리를 수행하였습니다.

로그인 기능 및 인증 처리

로그인은 기본적으로 OAuth와 자체 로그인을 개발하였습니다. OAuth의 경우 kakao, google 로그인을 지원하였으며 로그인 성공 시 jwt 토큰을 발급하여 클라이언트에게 전달하는 형태로 구성하였습니다. 해당 jwt 토큰의 인증은 apigateway server에서 global하게 처리하게 수행되며 현재는 apigateway에서 인증 완료할 경우 그 아래의 서버에 대한 접근과 사용이 허용되게 구성하였습니다. 현재는 클라이언트의 소셜 로그인 구현 문제로 자체 로그인으로 대체하였으며 자체 로그인의 경우 spring security를 이용하여 구현하였습니다.

MSA 구축

프로젝트는 장기적으로 확장될 계획이었고, 초기에는 핵심 기능만 구현한 뒤 점진적으로 새로운 기능을 추가해야 했습니다. 이에 따라 기존의 단일 구조로는 유연한 기능 확장과 독립적인 배포가 어렵다는 문제가 있었습니다. Spring Cloud 기반으로 MSA(Microservice Architecture)를 구축하여 서비스 간 결합도를 낮추고 유연한 구조를 만들었습니다.

Config Server, Game Server, Manage Server, API Gateway Server, Discovery Server, Monitoring Server를 Spring 프레임워크 기반으로 구축하고, 각 서버를 Docker 컨테이너로 배포했습니다. 서버 간 통신은 Spring Cloud의 OpenFeign을 활용하여 API 간 호출을 단순화했고, Spring Cloud CircuitBreaker를 적용해 장애 발생 시에도 안정적으로 요청을 처리할 수 있도록 했습니다. 공통 설정 파일을 중앙에서 관리하기 위해 Config Server를 도입했습니다. 각 서버는 기동 시 Config Server로부터 자신의 설정을 자동으로 불러오도록 구성하여 유지보수 효율을 높였습니다.

이를 통해 각 도메인의 독립적인 개발 및 배포가 가능해졌으며, 새로운 기능 추가 시에도 다른 서비스에 영향을 주지 않고 빠르게 확장할 수 있었습니다. 또한 통합된 설정 관리로 운영 효율성과 코드 일관성을 확보할 수 있었으며, 테스트와 유지보수 비용을 크게 줄일 수 있었습니다. 다만 CircuitBreaker와 OpenFeign의 경우 완벽히 구현하지는 못했습니다. 프로젝트를 진행하며 학습해야 할 범위가 넓었고, 기술에 대한 충분한 이해 없이 적용할 경우 오히려 장애로 이어질 수 있다는 점을 직접 경험했습니다. 이후 해당 기능들을 제거하고, 프로젝트의 안정성과 팀의 이해 수준에 맞는 기술 선택의 중요성을 깨달았습니다.

쿠타버스: 학교를 배경으로 한 메타버스 게임 제작

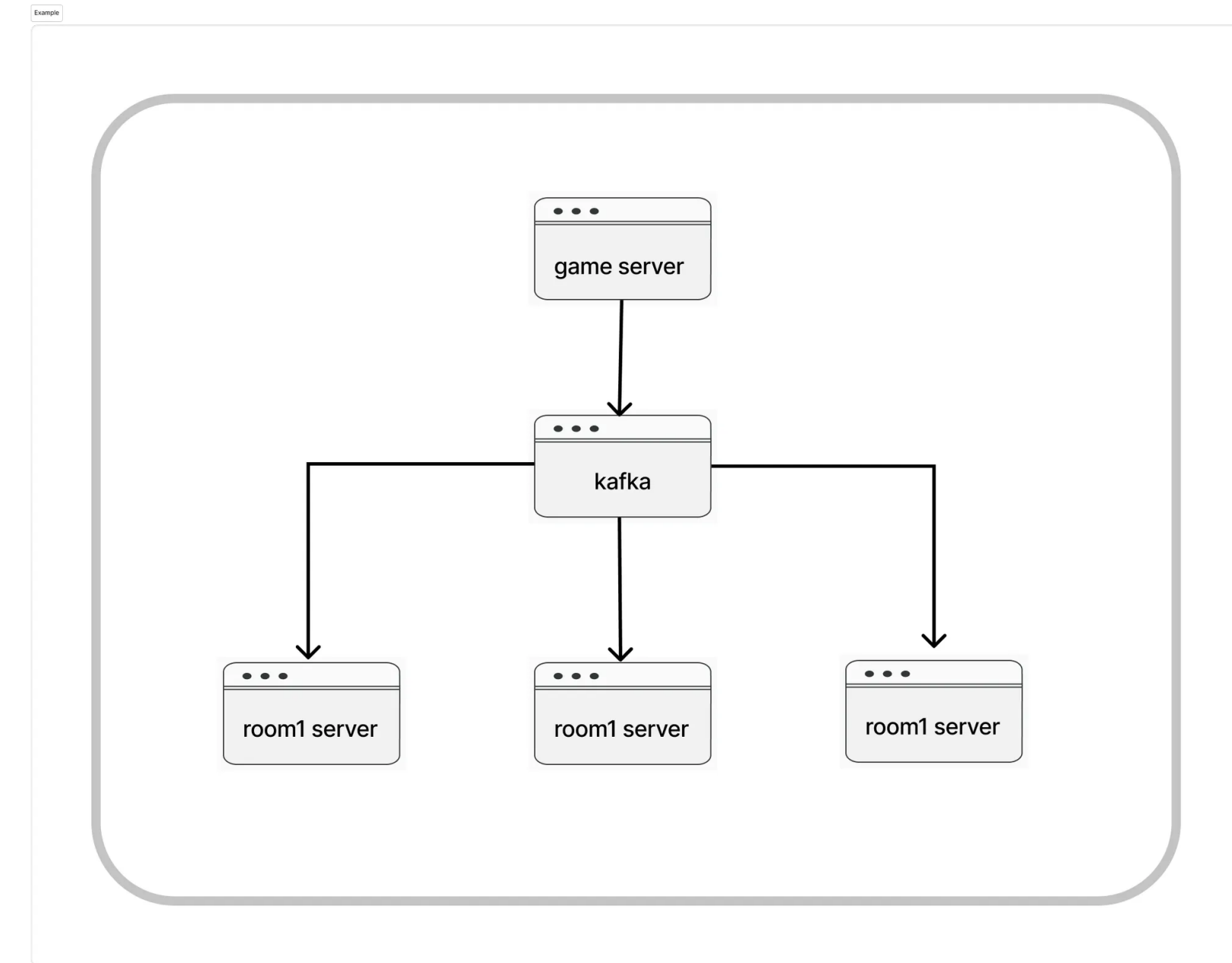
Github <https://github.com/KU-Taverse>

게임 서버 분산 처리

게임을 진행하는데 있어서 사용자 경험을 높일려면 30프레임 이상을 유지하는게 중요했습니다. 그 중 게임 내에서 30프레임 이상을 안정적으로 유지하기 위해서는 실시간 데이터 처리 속도가 매우 중요했습니다. 게임 서버는 미니게임 2개, 채팅, 맵 이동 등 다양한 요청을 동시에 처리해야 했고, 이에 따라 데이터 처리량이 급격히 증가하는 문제가 발생했습니다.

높은 동시성을 처리하기 위해 Spring WebFlux 기반으로 서버를 구현하여 비동기 요청을 효율적으로 처리했습니다. 또한 미니게임 처리 부하를 분산하기 위해 라운드 로빈 방식으로 요청을 여러 서버에 분산시켰습니다. 각 서버는 Kafka 토픽을 통해 데이터를 주고받으며, 방 생성·유저 이동·채팅 등의 정보를 비동기적으로 전달받도록 설계했습니다. 이를 통해 서버 간의 의존도를 낮추고, 분산 서버에서도 유저와 방 정보를 재연결 할 수 있도록 구현을 진행하였습니다.

이러한 구조를 통해 게임 내 실시간 응답성과 처리 안정성을 향상시킬 수 있었습니다. 다만 현재는 단순 라운드 로빈 기반의 분산 로직이 적용되어 있으며, 추후에는 부하 기반 동적 분산 로직으로 개선할 계획입니다. 또한 데이터 처리 속도를 높이기 위해 Spring WebFlux 를 사용했지만, 실제로는 Spring MVC와 큰 성능 차이가 없다는 점을 확인했습니다. 이를 통해 기술을 선택할 때 단순히 '비동기'나 '최신 기술'이라는 이유만으로 도입하기보다, 프로젝트의 요구사항과 환경에 적합한지를 먼저 검증하는 과정이 중요하다는 것을 깨달았습니다.



모니터링

아키텍처를 설계하고 서버를 관리하기 위해 모니터링 도구들을 활용하여 개발을 진행하였습니다. 모니터링 서버의 경우 서버의 로그 수집과 메트릭 수집을 통해 시각화하는 것을 수행하였습니다. 구현을 위해 grafana, loki, promtail, prometheus를 활용하여 구현하였으며 docker-compose를 통해 관리하였습니다.

분산 시스템을 활용한 웹 서버 실시간 로그 데이터 분석 및 이상 감지 시스템

Github <https://github.com/kamothi/spark-application>

어플리케이션 로그 분석 시스템

시스템 로그, 애플리케이션 로그, 비즈니스 로그 등등 다양한 레벨의 로그들이 존재하지만 여기서 분석하는 로그는 애플리케이션 레벨의 로그 데이터를 사용하여 팀이 정한 이상 탐지 기준으로 탐지하여 시각화하는 것을 목표로 하였습니다. 개발 초기에 다음의 목표를 세웠으며 해당 목표들을 성공적으로 수행할 수 있게 설계하였습니다.

- 실시간 로그 시각화 딜레이 2000ms 이하
- 분산 저장소(저장 서버가 없어도 모든 데이터를 얻을 수 있음)
- 초당 10,000개 이상의 요청 로그 처리
- 로그 유실률 0% 보장

역할

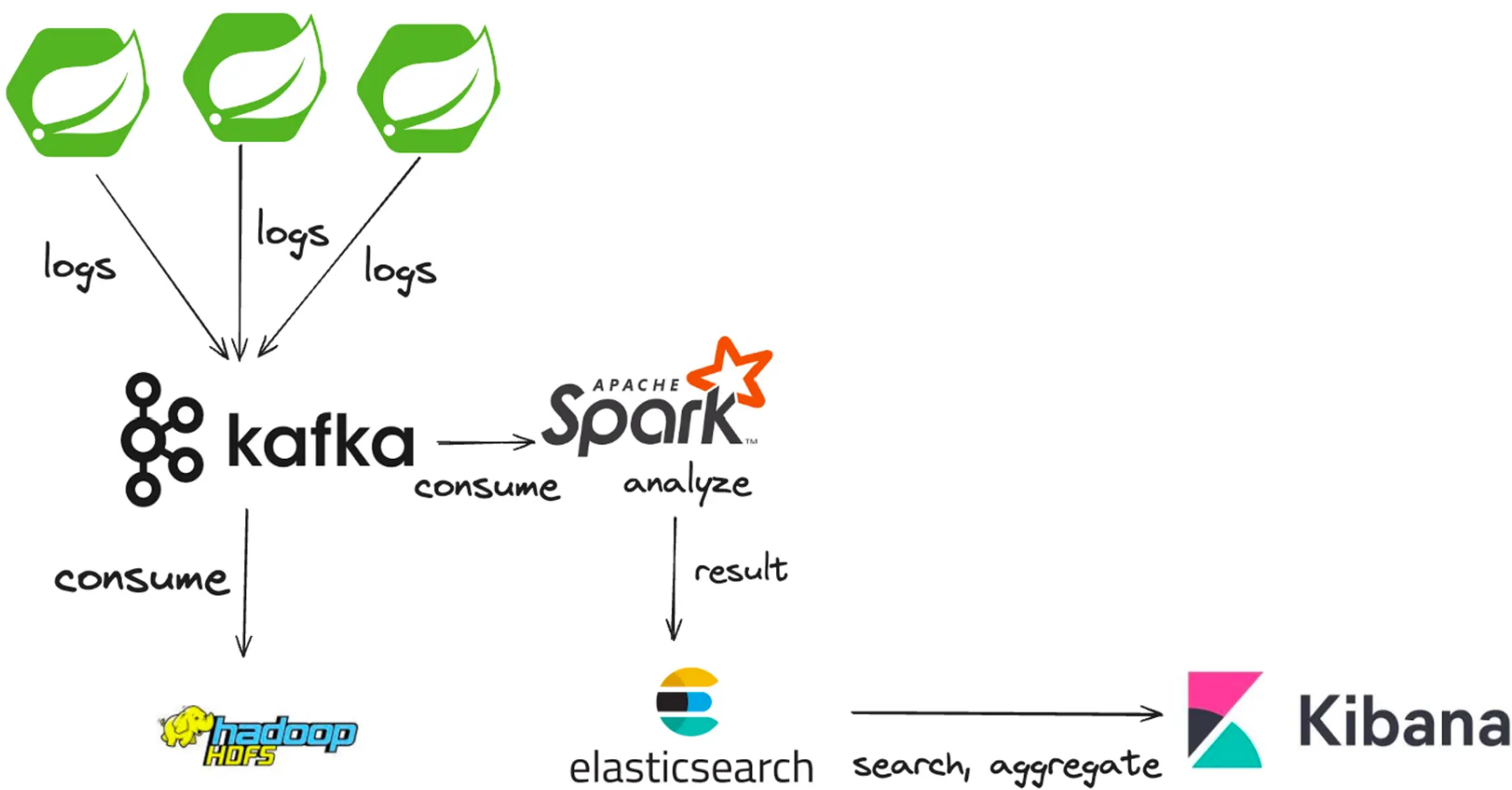
- Spring boot를 이용한 간단한 CRUD 설계
- Spring boot에서 발생하는 로그 데이터를 로그 파일로 저장
- 로그 파일의 데이터를 카프카 토픽에 메시지 전달
- 스파크 전처리 설계
- 스파크에서 전처리 된 데이터를 엘라스틱서치에 저장

기간

2024 04 ~ 2024.06

기술 스택

Spring Boot, Apache Spark, Kafka, Elastic Search, Kibana



구현 일지 및 기록

- 문제 정의 및 설계
- Spring boot 로그 설정
- 스파크 도입기 1
- 엘라스틱 서치 도입기
- 로그 분석 설정 및 문제 정의
- 가상 머신으로의 마이그레이션

분산 시스템을 활용한 웹 서버 실시간 로그 데이터 분석 및 이상 감지 시스템

Github <https://github.com/kamothi/spark-application>

로그 데이터 수집

Spring Boot 서버에서 발생하는 요청 데이터를 효율적으로 수집하고 분석을 목표로 잡았습니다. 수집 대상 정보는 요청 API, 포트, 유저 ID, Request Method, Request Parameters, Request Body 등 모든 API 요청에 공통적으로 필요한 정보였습니다.

모든 API 요청에 대해 로그를 자동으로 수집하기 위해 Spring AOP를 활용했습니다. 각 Controller에 들어오는 요청을 가로채어 필요한 정보를 추출하고 로그를 생성하도록 구현했습니다. 생성된 로그는 스케줄링을 통해 Kafka 토픽으로 전달되도록 구성하여, 이후 분석과 통합 처리에 활용할 수 있도록 설계했습니다.

이 구조를 통해 모든 서버에서 발생하는 요청 데이터를 일관성 있게 수집할 수 있었고, Kafka 기반의 로그 분석 시스템과 자연스럽게 연동될 수 있었습니다. 또한 AOP를 활용하여 코드 수정 없이도 모든 API 요청에 대한 로그 수집이 가능해 운영 효율성과 확장성을 확보할 수 있었습니다.

로그 데이터 전달

여러 Spring Boot 서버에서 발생하는 로그와 데이터를 효율적으로 수집 및 분석할 필요가 있었습니다. 서버별 데이터를 분산 처리하면서, 통합된 방식으로 로그를 분석하고 Elasticsearch에 전달할 수 있는 구조가 필요했습니다.

데이터 전달 방식으로 Kafka를 선택했습니다. 그 이유는 Spring과 Apache Spark 모두 Kafka를 지원하여 연동이 용이했습니다. 또한 기존 아키텍처에서 여러 서버의 로그를 토픽별로 분리하여 관리할 필요가 있었습니다. 구현 방식은 다음과 같습니다.

1. 각 Spring Boot 서버는 Producer로서 데이터를 Kafka 토픽에 등록합니다.
2. Apache Spark는 Consumer로 데이터를 가져와 로그를 분석하고, 처리 결과를 Elasticsearch로 전달합니다.

이를 통해 서버 간 데이터 수집과 분석을 효율적이고 확장 가능하게 구성할 수 있었습니다. Kafka 토픽 단위의 분산 구조로 로그 관리와 분석의 유연성도 확보할 수 있었습니다.

로그 분석 처리

로그 분석의 경우 정말 다양한 이상들에 대해 판단을 해야합니다. 초기 이상 탐지에 기준은 아래와 같이 잡았었습니다.

- 1분에 100번 이상의 요청이 들어온 경우 (DOS)
- 계속해서 요청이 실패하는 경우 - 브루트포스로 로그인 시도
- 요청 body 값을 확인하여 요청이 이상한 경우 - json injection
- 잘못된 Api path로의 요청이 감지 - 디렉토리 리스팅

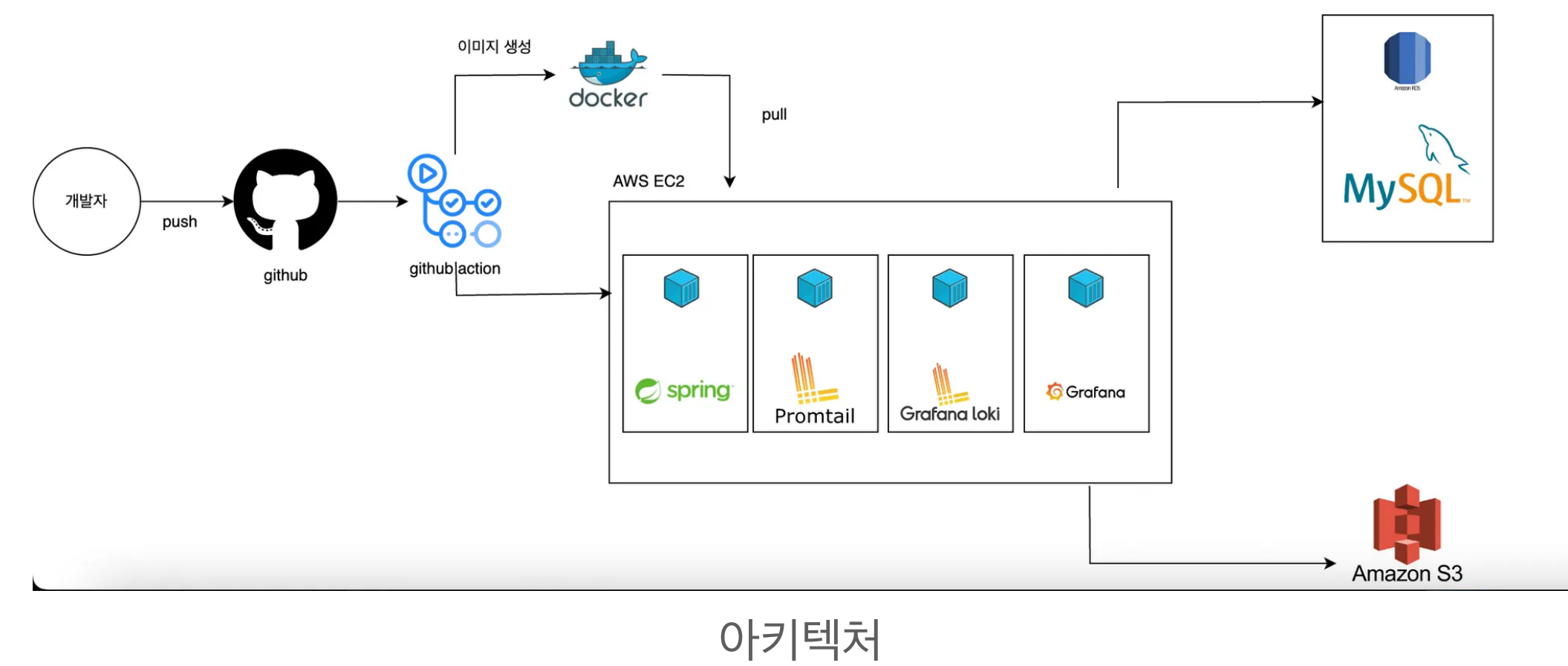
최종적으로 구현에 성공한 것은 sql injection과 json injection, DOS 에 대한 판단입니다. DOS에 대한 판단의 경우 특정 ip로 오는 요청들에 대해 kibana에서 alert 설정을 통해 1분에 100번 이상의 요청이 들어온 경우 slack으로 알림이 가게 설정을 하였습니다. 나머지 injection 판단의 경우 spark에서 처리하게 구성하였습니다. 일반적으로 spring boot 로그들에 대한 전처리 과정을 먼저 거치며 전처리를 거친 데이터들에 대해 injection 공격에 자주 사용되는 UNION에 대한 키워드와 주석문 그리고 세미콜론과 같은 특수 기호들이 존재하는지 유무를 판단하여 그에 대한 판별된 boolean형태의 정보를 담아 elasticsearch에 전달하게 구성하였습니다.

40 ~50대를 위한 건강 관리 웹 애플리케이션

챌린지 수행과 건강 진단을 통해 자신의 상태를 확인하고 자발적 건강 관리를 장려하는 것을 목표로하였습니다. 개발 스택으로는 spring boot, mysql, nginx를 활용하였으며 AWS의 ec2와 rds를 활용하여 배포를 하였습니다.

역할

- Spring Batch를 통한 건강 진단 분석 처리
- Grafana를 활용한 모니터링 세팅
- ChatGPT API를 활용한 건강 진단 분석 처리 구축
- Github Action과 Docker를 활용한 CI/CD 파이프라인 구축



기간

2024 03 ~ 2024.12

기술 스택

Spring Boot, MySql, Spring, Docker, AWS, Nginx, Grafana

구현 일지 및 기록

- 도커 다운으로 인한 자동화 스크립트 도입기
- 모니터링 도구 도입기
- 배치 처리 도입기 및 정리
- 쿼리 최적화 그거 어떻게 하는건데?

모니터링 및 알람

백엔드 기능 개발 중 의도치 않은 에러가 발생하는 경우가 있었습니다. 서버 로그를 실시간으로 확인하지 않으면 문제를 즉시 파악하기 어려워 빠른 대응이 어렵다는 문제가 있었습니다.

이 문제를 해결하기 위해 실시간 로그 모니터링 체계를 구축하여 에러 발생 시 즉시 대응할 수 있도록 설계했습니다. Spring Boot 서버의 로그 데이터를 Promtail, Loki, Grafana로 수집하였습니다. Grafana 알람 기능을 활용하여, 1분에 1건 이상의 에러 발생 시 Discord로 알림을 전송하게 구성하였습니다.

이를 통해 팀에서 개발 대응 속도와 서비스 안정성을 크게 향상시킬 수 있었습니다.

건강 진단 분석

건강 진단을 위해서는 여러 방식들이 존재하지만 비용적인 측면을 고려하여 ChatGPT API를 활용하여 구성하였습니다. 해당 api의 경우 GPT 버전마다 다양한 가격들이 책정되어 있으며 100명의 유저가 한 달 동안 매일 들어온다고 했을 경우 gpt-3.5 turbo가 대략 5만원 정도의 비용이 발생하는 것을 확인하고 해당 모델을 선정하여 사용하였습니다.

아키텍처 설계 및 ci/cd

docker와 github action을 통해 ci/cd를 구성하였습니다. 기본적으로 PR을 올릴 경우 테스트 코드들에 대한 테스트를 처리하게 구성하였으며 merge를 할 경우 자동으로 aws ec2에 배포 할 수 있게 구성하였습니다. ec2 1대에 총 4개의 컨테이너를 올리는 형태로 구성하였습니다.

건강 진단 배치 처리

일일 건강 진단과 달리 주간·월간 진단은 일일 진단 데이터를 모아 분석해야 했습니다. 이때, 배치 작업이 중간에 멈출 경우 데이터 손실과 가용성 문제가 발생할 수 있어 안정적인 처리가 필요했습니다.

Spring Batch를 활용하여 배치 작업을 Job 단위로 분리하고, 각 읽기, 쓰기, 저장 작업을 Chunk 단위로 처리하도록 설계했습니다. 매주 일요일 23시, 매월 마지막 날 23시에 작업 수행하게 세팅하였습니다. Spring Batch를 통해 중간에 작업이 중단되더라도 현재 Task부터 재실행 가능하며 데이터 손실 없이 작업을 이어갈 수 있었습니다.

이를 통해 주간·월간 건강 진단 데이터를 안정적이게 처리할 수 있게 되었으며, 단순 스케줄링 대비 가용성과 신뢰성을 크게 향상시켰습니다. 또한 배치 작업의 체계적 구조로 유지보수 및 확장성도 챙길 수 있었습니다.

스크립트 파일을 이용한 컨테이너 다운 문제 해결

Docker로 서버를 배포할 때, 테스트가 완벽히 끝나지 않은 상태에서 배포하면 컨테이너가 예기치 않게 종료되는 문제가 발생했습니다. GitHub Actions에서는 배포가 정상으로 처리되었지만, 실제 운영 환경에서는 다양한 이유로 컨테이너가 강제 종료될 수 있으며, 이를 실시간으로 모니터링하지 않으면 문제를 즉시 파악하기 어려웠습니다.

컨테이너 상태를 자동으로 체크하고 문제 발생 시 즉시 대응할 수 있도록 스크립트 파일을 활용한 모니터링 및 자동 복구 시스템을 구현했습니다.

- 컨테이너 상태를 주기적으로 확인
- 종료된 경우 Discord 알림 발송
- 자동으로 컨테이너를 재기동

이를 통해 배포 후 발생하는 컨테이너 다운 문제를 실시간으로 감지하고 자동 복구할 수 있게 되었으며, 운영 안정성이 크게 향상되었습니다. 또한 수동 모니터링 부담을 줄여 운영 효율성과 신속한 대응 능력을 확보할 수 있었습니다.